



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 08

NOMBRE COMPLETO: Camacho Ignacio Violeta

N° de Cuenta: 319061345

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 06

SEMESTRE 2026-2

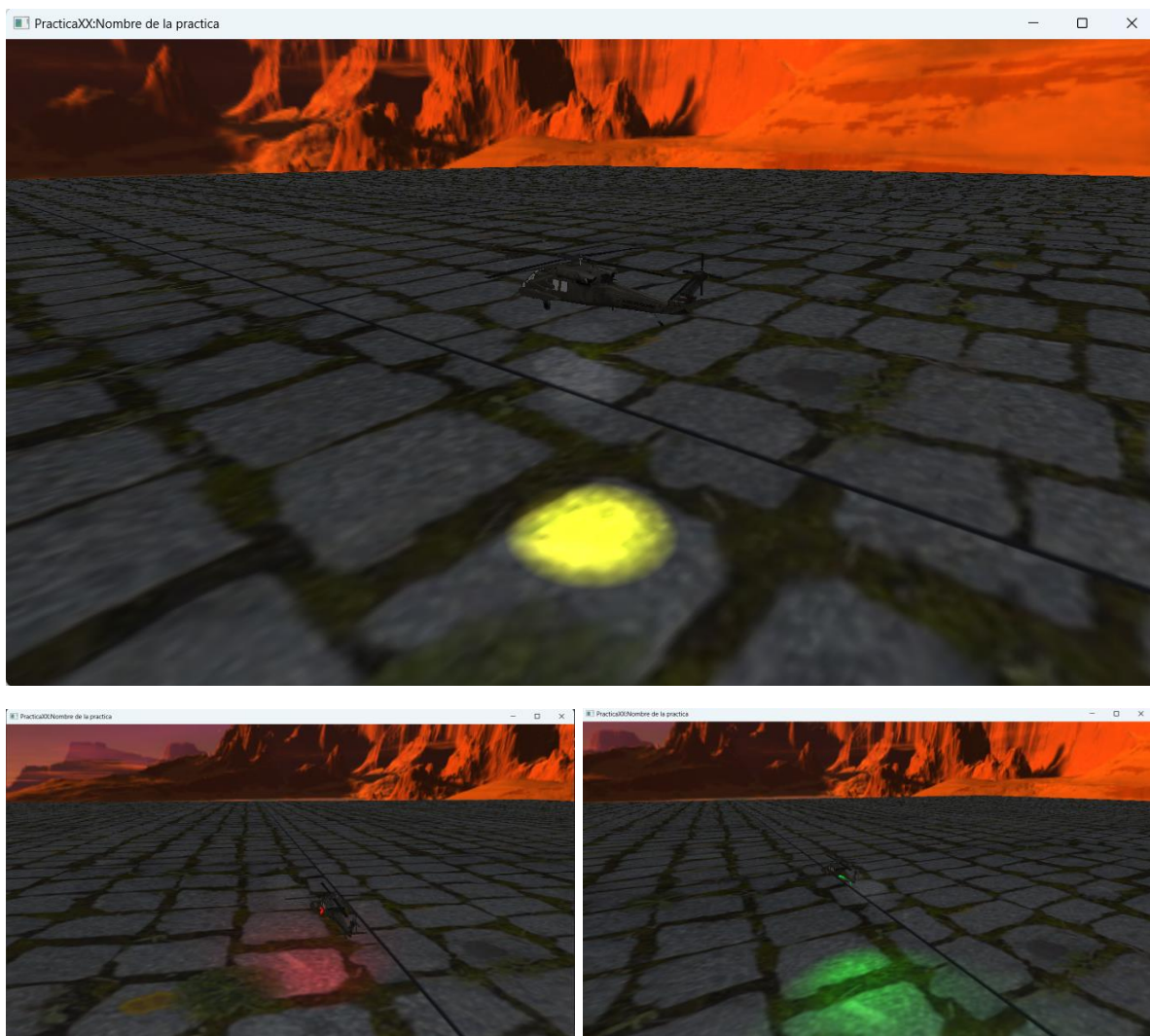
FECHA DE ENTREGA LÍMITE: 16-04-2026

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

1. Agregar luz de tipo spotlight para el helicóptero de tal forma que al avanzar (mover con teclado hacia dirección de X negativa) ilumine con un spotlight en el suelo hacia adelante y al retroceder (mover con teclado hacia dirección de X positiva) ilumine con un spotlight el suelo hacia atrás. Son dos spotlights diferentes que se prenderán y apagarán de acuerdo con la dirección en la que esté moviéndose el helicóptero.



Para el desarrollo de este ejercicio se creó un arreglo de 3 luces spotlight con características similares a la que se había usado antes para el spotlight amarillo

del helicóptero solamente que reduciendo el ángulo de cono, la atenuación y para demostrar el cambio entre los elementos del arreglo de luces, diferentes colores.

En la clase window ya se tenía el manejo del helicóptero para hacerlo avanzar hacia el eje x negativo y retroceder en el eje x positivo, aquí se introdujo una variable indiceHeli a cual indica qué spotlight debe activarse, el 0 que indica la luz hacia adelante, el 1 que es la luz hacia atrás o el 3 que es la luz apuntando hacia el suelo por defecto.

```
400 SpotLight farosHelicoptero[3];
```

```
435 spotLightCount++; //0 linterna
436 spotLightCount++; //1 faros d colores
437 spotLightCount++; //2 slot vacio para algo mas
438 spotLightCount++; //3 luces helicoptero
```

```
486 //----- luz helicoptero -----
487
488 farosHelicoptero[0] = SpotLight(
489     1.0f, 0.0f, 0.0f, // color rojo
490     0.3f, 0.3f, // intensidad
491     0.0f, 0.0f, 0.0f, // posición inicial
492     -5.0f, -5.0f, 0.0f, // hacia frente
493     1.0f, 0.7f, 1.8f, // atenuación (alcance bajo)
494     5.0f // ángulo (cono)
495 ); //
496
497 farosHelicoptero[1] = SpotLight(
498     0.0f, 1.0f, 0.0f, // color verde
499     0.3f, 0.3f, // intensidad
500     0.0f, 0.0f, 0.0f, // posición inicial
501     +5.0f, -5.0f, 0.0f, // hacia atras **cambiar por pecra
502     1.0f, 0.7f, 1.8f, // atenuación (alcance medio)
503     5.0f // ángulo (cono)
504 ); //
505
506 farosHelicoptero[2] = SpotLight(
507     1.0f, 1.0f, 0.0f, // color amarillo
508     0.5f, 0.8f, // intensidad
509     0.0f, 0.0f, 0.0f, // posición inicial
510     0.0f, -5.0f, 0.0f, // hacia el piso
511     0.1f, 0.05f, 0.01f, // atenuación (alcance medio)
512     20.0f // ángulo (cono)
513 ); //
```

```

34 | // HELICÓPTERO ADELANTE
35 | if (key == GLFW_KEY_J)
36 | {
37 |     theWindow->muevex2 -= 0.5f;
38 |     theWindow->indiceHeli = 0; // avanza luz hacia adelante
39 | }
40 |
41 | // HELICÓPTERO ATRÁS |
42 | if (key == GLFW_KEY_K)
43 | {
44 |     theWindow->muevex2 += 0.5f;
45 |     theWindow->indiceHeli = 1; // retrocede luz hacia atrás
46 | }
47 |
48 | if ((key == GLFW_KEY_J || key == GLFW_KEY_K) && action == GLFW_RELEASE) //luz por defecto
49 | {
50 |     theWindow->indiceHeli = 2; // quieto
51 | }

```

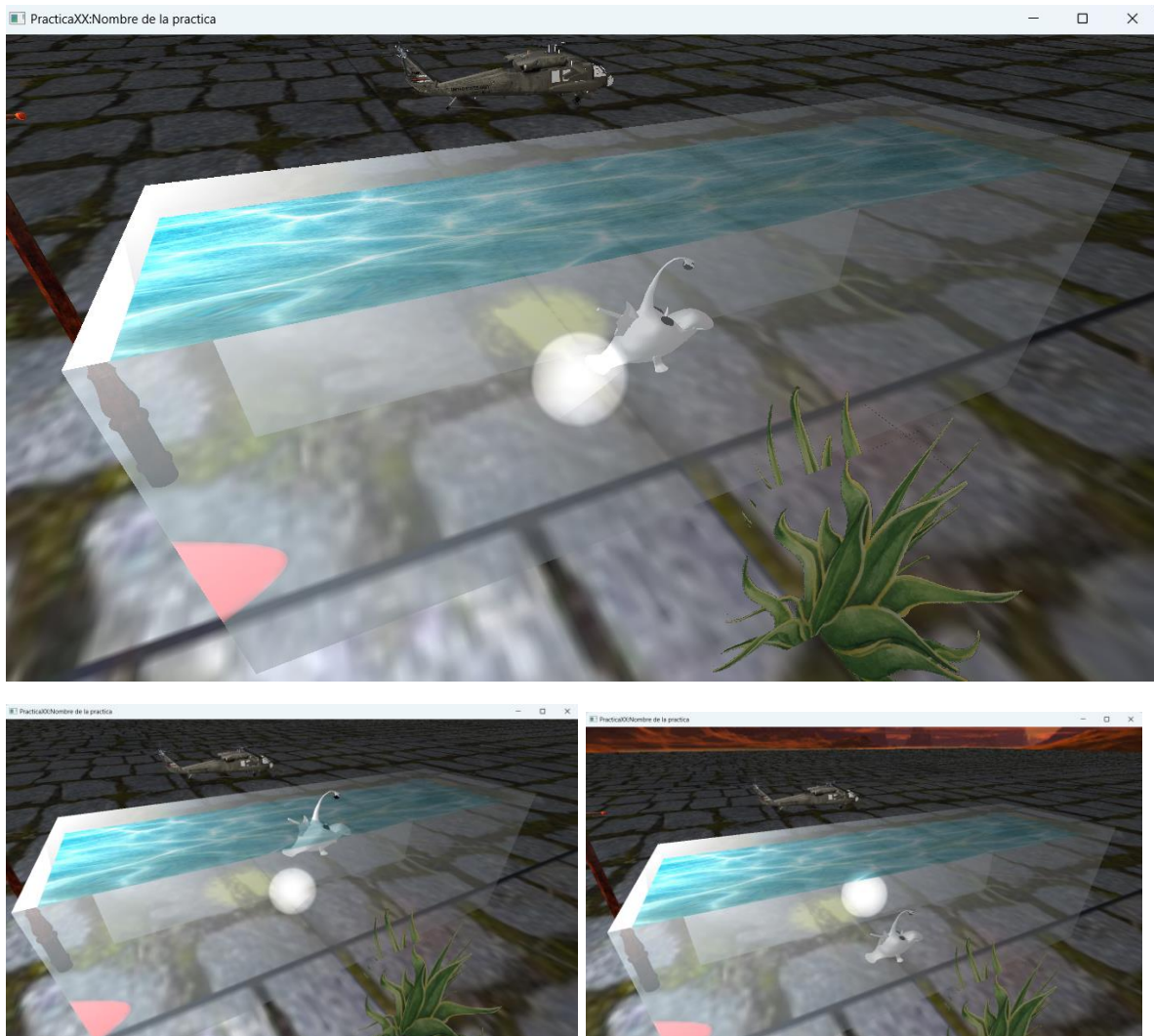
Para evitar que la dirección del spotlight se sobrescribiera la dirección del spotlight se define también según el estado de indiceHeli

```

670 | // dirección según el índice del helicóptero
671 | glm::vec3 dirHeli;
672 | if (mainWindow.getIndiceHeli() == 0)
673 |     dirHeli = glm::vec3(-1.0f, -1.0f, 0.0f); // adelante
674 | else if (mainWindow.getIndiceHeli() == 1)
675 |     dirHeli = glm::vec3(1.0f, -1.0f, 0.0f); // atrás
676 | else
677 |     dirHeli = glm::vec3(0.0f, -1.0f, 0.0f); // quieto, hacia abajo
678 |
679 | farosHelicoptero[mainWindow.getIndiceHeli()].setFlash(posHeli, dirHeli);
680 | spotLights[3] = farosHelicoptero[mainWindow.getIndiceHeli()];
681 |

```

2.- Crear o instanciar una pecera (traslúcida) con las normales hacia adentro y en la parte superior imagen de agua e instanciar dentro de la pecera al pez abisal con movimiento de teclado para subir y bajar en diagonal.

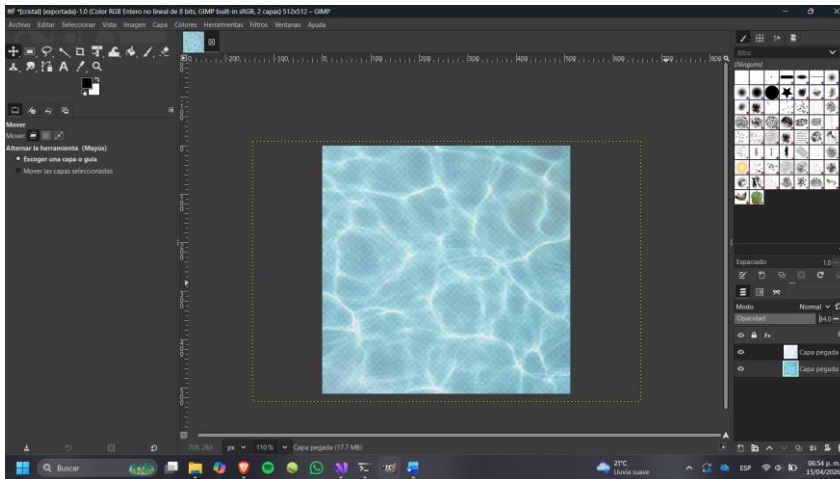


Se creó una pecera a partir de un prisma rectangular modificando el código existente del dado de 6 caras pero separando la cara superior en otra instancia para ponerle una textura distinta mientras que el resto de la pecera utiliza una textura de vidrio, ajustando sus dimensiones, las normales se orientaron hacia el interior para que la iluminación afecte correctamente las caras

Para lograr el efecto translúcido se utilizó blending junto con una textura con canal alfa que se ajustó en gimp permitiendo que la pecera se vea transparente, además se cuidó el orden de renderizado para que los objetos transparentes se dibujen después de los sólidos por eso la pecera quedó después de los modelos pero antes del agave.

Dentro de la pecera se instanció un modelo de pez abisal que se convirtió primero a .obj en blender el cual se escaló y se rotó para que quedara orientado con la pecera, su movimiento se controla mediante el teclado con las teclas I y M,

permitiendo que el pez suba y baje, generando un desplazamiento en diagonal dentro de la pecera.



```
89 Texture aguaTexture;  
90 Texture cristalTexture;
```

```
241 void CrearDado()  
242 {  
243     unsigned int cubo_indices[] = {  
244         // right  
245         0, 1, 2,  
246         2, 3, 0,  
247  
248         // left  
249         8, 9, 10,  
250         10, 11, 8,  
251  
252         // back  
253         12, 13, 14,  
254         14, 15, 12,  
255         // bottom  
256         16, 17, 18,  
257         18, 19, 16,  
258  
259         // front  
260         4, 5, 6,  
261         6, 7, 4,
```

```

263 }; GLfloat cubo_vertices[] = {
264     //orientacion del helicoptero frente + x      parte trasera - x
265     // right +z
266     //x      y      z      S      T      NX      NY      NZ
267     -1.5f, -0.5f, 0.5f, 0.26f, 0.34f, 0.0f, 0.0f, -1.0f, //0
268     1.5f, -0.5f, 0.5f, 0.49f, 0.34f, 0.0f, 0.0f, -1.0f, //1
269     1.5f, 0.5f, 0.5f, 0.49f, 0.66f, 0.0f, 0.0f, -1.0f, //2
270     -1.5f, 0.5f, 0.5f, 0.26f, 0.66f, 0.0f, 0.0f, -1.0f, //3
271     // front +x
272     //x      y      z      S      T
273     1.5f, -0.5f, 0.5f, 0.0f, 0.0f, -1.0f, 0.0f, 0.0f, //4
274     1.5f, -0.5f, -0.5f, 1.0f, 0.0f, -1.0f, 0.0f, 0.0f, //5
275     1.5f, 0.5f, -0.5f, 1.0f, 1.0f, -1.0f, 0.0f, 0.0f, //6
276     1.5f, 0.5f, 0.5f, 0.0f, 1.0f, -1.0f, 0.0f, 0.0f, //7
277
278     // left -z
279     -1.5f, -0.5f, -0.5f, 0.0f, 0.0f, 0.0f, 0.0f, 1.0f, //8
280     1.5f, -0.5f, -0.5f, 1.0f, 0.0f, 0.0f, 0.0f, 1.0f, //9
281     1.5f, 0.5f, -0.5f, 1.0f, 1.0f, 0.0f, 0.0f, 1.0f, //10
282     -1.5f, 0.5f, -0.5f, 0.0f, 1.0f, 0.0f, 0.0f, 1.0f, //11
283
284     // back -x
285     //x      y      z      S      T
286     -1.5f, -0.5f, -0.5f, 0.0f, 0.0f, 1.0f, 0.0f, 0.0f, //12
287     -1.5f, -0.5f, 0.5f, 1.0f, 0.0f, 1.0f, 0.0f, 0.0f, //13
288     -1.5f, 0.5f, 0.5f, 1.0f, 1.0f, 1.0f, 0.0f, 0.0f, //14
289     -1.5f, 0.5f, -0.5f, 0.0f, 1.0f, 1.0f, 0.0f, 0.0f, //15
290
291     // bottom
292     //x      y      z      S      T
293     -1.5f, -0.5f, 0.5f, 0.0f, 0.0f, 0.0f, 1.0f, 0.0f, //16
294     1.5f, -0.5f, 0.5f, 1.0f, 0.0f, 0.0f, 1.0f, 0.0f, //17
295     1.5f, -0.5f, -0.5f, 1.0f, 1.0f, 0.0f, 1.0f, 0.0f, //18
296     -1.5f, -0.5f, -0.5f, 0.0f, 1.0f, 0.0f, 1.0f, 0.0f, //19
297
298 };

```

```

306 void CrearTapa()
307 {
308     unsigned int indices[] = {
309         0, 1, 2,
310         2, 3, 0
311     };
312
313     GLfloat vertices[] = {
314         //UP
315         // x      y      z      S      T      NX      NY      NZ
316         -1.5f, 0.4f, 0.5f, 0.0f, 0.0f, 0.0f, 1.0f, 0.0f,
317         1.5f, 0.4f, 0.5f, 1.0f, 0.0f, 0.0f, 1.0f, 0.0f,
318         1.5f, 0.4f, -0.5f, 1.0f, 1.0f, 0.0f, 1.0f, 0.0f,
319         -1.5f, 0.4f, -0.5f, 0.0f, 1.0f, 0.0f, 1.0f, 0.0f
320     };
321
322     Mesh* tapa = new Mesh();
323     tapa->CreateMesh(vertices, indices, 32, 6);
324     meshList.push_back(tapa);
325 }
326

```

```

334 CrearDado();
335 CrearTapa();

```

```

354     aguaTexture = Texture("Texturas/agua.png"); //textura agua-----
355     aguaTexture.LoadTextureA();
356
357     cristalTexture = Texture("Texturas/cristal.png"); //textura cristal-----
358     cristalTexture.LoadTextureA();
359
360
361
362
363
364     //----- Ejercicio 2 pecera -----
365     glEnable(GL_BLEND);
366     glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
367     model = glm::mat4(1.0); //pecera
368     model = glm::translate(model, glm::vec3(0.0f, 2.0f, 1.0f));
369     model = glm::scale(model, glm::vec3(5.0f, 5.0f, 5.0f));
370     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
371
372     cristalTexture.UseTexture(); // textura
373     meshList[4]->RenderMesh();
374
375     //---
376
377     model = glm::mat4(1.0); //tapa
378     model = glm::translate(model, glm::vec3(0.0f, 2.0f, 1.0f));
379     model = glm::scale(model, glm::vec3(5.0f, 5.0f, 5.0f));
380     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
381
382     aguaTexture.UseTexture(); // textura
383     meshList[5]->RenderMesh(); // tapa
384     glDisable(GL_BLEND);
385     //-----

```

3.- Agregar una luz de tipo puntual de color azul ligada al bulbo del pez que puedan prender y apagar de forma independiente con teclado tanto la luz de la lámpara de su práctica 7 como esta luz (la luz de la lámpara debe de ser puntual, si la crearon spotlight en su reporte 7 tienen que cambiarla a luz puntual), de tal forma que se pueda ver: las 2 luces apagadas, las 2 luces prendidas, una luz prendida y una luz apagada y viceversa.

4. Agregar un spotlight (que no sea luz de color blanco ni azul) que parta del bulbo del pez y que ilumine en cualquier dirección dentro de la pecera al presionar teclas asignadas al eje x, eje y y eje z.

Liga de descarga del modelo de pez editado: <https://sketchfab.com/3d-models/pez-abisal-5ca9b50c85424d52a29f47f065f05ab9>

2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla

Uno de los principales inconvenientes fue que la pecera no se mostraba translúcida a pesar de haber exportado correctamente la textura con canal alfa, este problema se resolvió activando el uso de blending en OpenGL (`glEnable(GL_BLEND)`) y asegurando que el shader utilizara el valor alpha de la textura, además de renderizar la pecera después de los objetos sólidos para evitar conflictos con el buffer de profundidad.

Otro problema fue que la luz tipo spotlight del helicóptero tenía un alcance excesivo, iluminando prácticamente toda la escena, esto se solucionó ajustando los parámetros de atenuación (constante, lineal y cuadrática), logrando que la luz tuviera un alcance más realista y controlado, también fue necesario ajustar el ángulo del cono de luz para evitar que el haz fuera demasiado estrecho o intenso.

3.- Conclusión:

- a. Los ejercicios del reporte: Complejidad, Explicación.
Los ejercicios presentaron una complejidad alta, ya que implicaron la integración de varios conceptos importantes de gráficos por computadora, como iluminación (especialmente spotlight), manejo de texturas, uso de blending para transparencia y transformaciones jerárquicas, aunque cada parte por separado es comprensible, combinarlas correctamente dentro del ciclo de renderizado requirió atención al detalle y comprensión del flujo de OpenGL.
- b. Comentarios generales: Faltó explicar a detalle, ir más lento en alguna explicación, otros comentarios y sugerencias para mejorar desarrollo de la práctica
Si bien son ejercicios que no se alejan mucho de lo que ya hemos visto el combinar tantos aspectos diferentes en cada punto aumentó significativamente su dificultad ya que cada uno requería mucho tiempo y atención al detalle debido a que hacía varias cosas al mismo tiempo, como sugerencia, incluir ejemplos más guiados paso a paso ayudaría a reforzar el aprendizaje y más sencillos en términos de acciones a realizar por ejercicio.
- c. Conclusión
La práctica permitió comprender cómo implementar distintos tipos de iluminación en una escena 3D, así como el uso de texturas y transparencia para lograr efectos más realistas, además, se reforzó el uso de transformaciones y control por teclado para generar interacción en iluminación en tiempo real.

1. Bibliografía en formato APA

- Freepik. (s.f.). Textura de agua de piscina. Recuperado de <https://www.freepik.es/foto-gratis/primer-plano-textura-agua->

piscina_28741694.htm#fromView=keyword&page=1&position=0&uuid=82b16d89-2be4-44d1-84e0-3fec6db5b694&query=Water+texture

- Shutterstock. (s.f.). Mirror surface background transparent glass showcase. Recuperado de <https://www.shutterstock.com/es/image-vector/mirror-surface-background-transparent-glass-showcase-2567439943>
- LearnOpenGL. (s.f.). Light casters. Recuperado de <https://learnopengl.com/Lighting/Light-casters>
- LearnOpenGL. (s.f.). Multiple lights. Recuperado de <https://learnopengl.com/Lighting/Multiple-lights>
- Ogldev. (s.f.). Tutorial 21: Spot Light. Recuperado de <https://www.ogldev.org/www/tutorial21/tutorial21.html>
- Khronos Group. (s.f.). OpenGL spotlight effect. Recuperado de <https://www.opengl.org/archives/resources/code/samples/advanced/advanced96/node64.html>
- Microsoft Docs. (s.f.). Attenuation and spotlight factor. Recuperado de <https://github.com/MicrosoftDocs/windows-dev-docs/blob/docs/uwp/graphics-concepts/attenuation-and-spotlight-factor.md>